인코더 아키텍처 (Encoder Architecture) 구현

개요

트랜스포머의 인코더는 입력 시퀀스를 처리하여 풍부한 표현(representation)을 생성하는 핵심 구성 요소입니다. 인코더는 멀티헤드 어텐션, 피드포워드 네트워크, 잔차 연결, 그리고 레이어 정규화를 조합하여 강력한 시퀀스 모델링 능력을 제공합니다.

인코더 레이어의 구조

1. 전체 아키텍처 개요

인코더 레이어는 다음과 같은 구성 요소들로 이루어져 있습니다:

```
입력 → 멀티헤드 어텐션 → 잔차 연결 & 레이어 정규화 → 피드포워드 네트워크 → 잔차 연결 & 레
```

2. 각 구성 요소의 역할

멀티헤드 셀프 어텐션 (Multi-Head Self-Attention)

- **목적**: 시퀀스 내 모든 위치 간의 관계를 모델링
- **특징**: 입력 시퀀스의 각 토큰이 다른 모든 토큰과 상호작용
- **효과**: 장거리 의존성과 복잡한 패턴 포착

피드포워드 네트워크 (Feed-Forward Network)

- **목적**: 각 위치에서 독립적으로 비선형 변환 수행
- **구조**: 두 개의 선형 변환과 ReLU 활성화 함수

- **효과**: 표현력 증대와 복잡한 특징 학습

잔차 연결 (Residual Connection)

- **목적**: 그래디언트 소실 문제 해결과 학습 안정성 향상
- **구현**: `output = layer(input) + input`
- **효과**: 깊은 네트워크에서도 안정적인 학습 가능

레이어 정규화 (Layer Normalization)

- **목적**: 각 레이어의 출력을 정규화하여 학습 안정성 향상
- **특징**: 배치 정규화와 달리 시퀀스 길이에 무관하게 동작
- **효과**: 빠른 수렴과 안정적인 훈련

잔차 연결과 레이어 정규화의 필요성

1. 잔차 연결의 중요성

그래디언트 소실 문제 해결

```
# 잔차 연결 없이 (문제가 있는 경우)
def without_residual(x, layer):
return layer(x)

# 잔차 연결 적용 (올바른 방법)
def with_residual(x, layer):
return layer(x) + x # 입력을 출력에 직접 더함
```

그래디언트 흐름 개선

잔차 연결이 있을 때 역전파 과정에서:

```
∂L/∂x = ∂L/∂output × (∂layer(x)/∂x + 1)
```

여기서 `+1` 항이 그래디언트가 직접 흘러갈 수 있는 경로를 제공합니다.

2. 레이어 정규화의 필요성

배치 정규화 vs 레이어 정규화

```
# 배치 정규화 (시퀀스 모델에 부적합)
# 배치 내 다른 샘플들의 통계를 사용 → 시퀀스 길이 의존성 문제

# 레이어 정규화 (시퀀스 모델에 적합)
# 각 샘플의 특징 차원에 대해서만 정규화 → 시퀀스 길이 무관
```

Pre-LN vs Post-LN 구조

Post-LN (원본 트랜스포머):

```
output = LayerNorm(x + SubLayer(x))
```

Pre-LN (개선된 버전):

```
output = x + SubLayer(LayerNorm(x))
```

Pre-LN이 더 안정적인 학습을 제공하는 것으로 알려져 있습니다.

완전한 인코더 레이어 클래스 구현

```
import torch

import torch.nn as nn

import torch.nn.functional as F

import math


class MultiHeadAttention(nn.Module):

"""멀티헤드 어텐션 구현 (이전 문서에서 구현한 클래스)"""

def  __init__(self, d_model, num_heads, dropout=0.1):

super(MultiHeadAttention, self).__init__()

assert d_model % num_heads == 0

self.d_model = d_model

self.num_heads = num_heads

self.d_k = d_model // num_heads

self.w_q = nn.Linear(d_model, d_model)

self.w_k = nn.Linear(d_model, d_model)

self.w_v = nn.Linear(d_model, d_model)

self.w_o = nn.Linear(d_model, d_model)

self.dropout = nn.Dropout(dropout)

def  scaled_dot_product_attention(self, query, key, value, mask=None):

d_k = query.size(-1)

scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
```

```
if mask is not None:
scores = scores.masked_fill(mask == 0, -1e9)
attention_weights = F.softmax(scores, dim=-1)
attention_weights = self.dropout(attention_weights)
output = torch.matmul(attention_weights, value)
return output, attention_weights
def  forward(self, query, key, value, mask=None):
batch_size, seq_len = query.size(0), query.size(1)
Q = self.w_q(query).view(batch_size, seq_len, self.num_heads, self.d_k).transpos
K = self.w_k(key).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(
V = self.w_v(value).view(batch_size, seq_len, self.num_heads, self.d_k).transpos
attention_output, attention_weights = self.scaled_dot_product_attention(Q, K, V,
attention_output = attention_output.transpose(1, 2).contiguous().view(
batch_size, seq_len, self.d_model
)
output = self.w_o(attention_output)
return output, attention_weights


class PositionwiseFeedForward(nn.Module):
"""
포지션별 피드포워드 네트워크
Args:
d_model: 모델의 차원
```

```
d_ff: 피드포워드 네트워크의 내부 차원 (보통 d_model의 4배)

dropout: 드롭아웃 비율

"""

def  __init__(self, d_model, d_ff, dropout=0.1):

super(PositionwiseFeedForward, self).__init__()

# 두 개의 선형 변환 레이어

self.linear1 = nn.Linear(d_model, d_ff)

self.linear2 = nn.Linear(d_ff, d_model)

# 드롭아웃

self.dropout = nn.Dropout(dropout)

def  forward(self, x):

"""

순전파

Args:

x: [batch_size, seq_len, d_model]

Returns:

output: [batch_size, seq_len, d_model]

"""

# 첫 번째 선형 변환 + ReLU 활성화

x = self.linear1(x)

x = F.relu(x)

x = self.dropout(x)

# 두 번째 선형 변환
```

```
x = self.linear2(x)

return x


class EncoderLayer(nn.Module):

"""

트랜스포머 인코더 레이어

구조:

1. 멀티헤드 셀프 어텐션

2. 잔차 연결 + 레이어 정규화

3. 피드포워드 네트워크

4. 잔차 연결 + 레이어 정규화

Args:

d_model: 모델의 차원

num_heads: 어텐션 헤드의 수

d_ff: 피드포워드 네트워크의 내부 차원

dropout: 드롭아웃 비율

"""

def  __init__(self, d_model, num_heads, d_ff, dropout=0.1):

super(EncoderLayer, self).__init__()

# 멀티헤드 어텐션

self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)

# 피드포워드 네트워크

self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)
```

```
# 레이어 정규화 (두 개의 서브레이어용)
self.norm1 = nn.LayerNorm(d_model)
self.norm2 = nn.LayerNorm(d_model)
# 드롭아웃
self.dropout = nn.Dropout(dropout)
def  forward(self, x, mask=None):
"""
인코더 레이어 순전파
Args:
x: [batch_size, seq_len, d_model] - 입력 시퀀스
mask: 어텐션 마스크 (패딩 토큰 마스킹용)
Returns:
output: [batch_size, seq_len, d_model] - 출력 시퀀스
attention_weights: 어텐션 가중치
"""
# 1. 멀티헤드 셀프 어텐션 + 잔차 연결 + 레이어 정규화
# Post-LN 구조: LayerNorm(x + SubLayer(x))
attention_output, attention_weights = self.self_attention(x, x, x, mask)
attention_output = self.dropout(attention_output)
x = self.norm1(x + attention_output) # 잔차 연결 + 레이어 정규화
# 2. 피드포워드 네트워크 + 잔차 연결 + 레이어 정규화
ff_output = self.feed_forward(x)
ff_output = self.dropout(ff_output)
```

```
output = self.norm2(x + ff_output) # 잔차 연결 + 레이어 정규화

return output, attention_weights


class EncoderLayerPreLN(nn.Module):

"""

Pre-LN 구조의 인코더 레이어 (개선된 버전)

구조:

1. 레이어 정규화 + 멀티헤드 셀프 어텐션 + 잔차 연결

2. 레이어 정규화 + 피드포워드 네트워크 + 잔차 연결

"""

def  __init__(self, d_model, num_heads, d_ff, dropout=0.1):

super(EncoderLayerPreLN, self).__init__()

self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)

self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

self.norm1 = nn.LayerNorm(d_model)

self.norm2 = nn.LayerNorm(d_model)

self.dropout = nn.Dropout(dropout)

def  forward(self, x, mask=None):

"""

Pre-LN 인코더 레이어 순전파

Pre-LN 구조: x + SubLayer(LayerNorm(x))

"""

# 1. 레이어 정규화 + 멀티헤드 어텐션 + 잔차 연결
```

```
norm_x = self.norm1(x)

attention_output, attention_weights = self.self_attention(norm_x, norm_x, norm_x

attention_output = self.dropout(attention_output)

x = x + attention_output # 잔차 연결

# 2. 레이어 정규화 + 피드포워드 + 잔차 연결

norm_x = self.norm2(x)

ff_output = self.feed_forward(norm_x)

ff_output = self.dropout(ff_output)

output = x + ff_output # 잔차 연결

return output, attention_weights
```

6개 레이어 스택의 의미와 효과

1. 계층적 표현 학습

트랜스포머 인코더는 6개의 동일한 레이어를 스택으로 쌓아 사용합니다. 각 레이어는 점진적으로 더 복잡하고 추상적인 표현을 학습합니다:

```
class TransformerEncoder(nn.Module):

"""

완전한 트랜스포머 인코더 (6개 레이어 스택)

Args:

num_layers: 인코더 레이어의 수 (기본값: 6)

d_model: 모델의 차원
```

```
num_heads: 어텐션 헤드의 수

d_ff: 피드포워드 네트워크의 내부 차원

dropout: 드롭아웃 비율

"""

def  __init__(self, num_layers=6, d_model=512, num_heads=8, d_ff=2048, dropout=0

super(TransformerEncoder, self).__init__()

self.num_layers = num_layers

self.d_model = d_model

# 6개의 동일한 인코더 레이어 생성

self.layers = nn.ModuleList([

EncoderLayer(d_model, num_heads, d_ff, dropout)

for _ in  range(num_layers)

])

# 최종 레이어 정규화 (Pre-LN 구조에서 필요)

self.final_norm = nn.LayerNorm(d_model)

def  forward(self, x, mask=None):

"""

인코더 순전파

Args:

x: [batch_size, seq_len, d_model] - 입력 임베딩 + 포지셔널 인코딩

mask: 패딩 마스크

Returns:

output: [batch_size, seq_len, d_model] - 인코더 출력
```

```
all_attention_weights: 모든 레이어의 어텐션 가중치
"""
all_attention_weights = []
# 각 레이어를 순차적으로 통과
for i, layer in  enumerate(self.layers):
x, attention_weights = layer(x, mask)
all_attention_weights.append(attention_weights)
# 중간 레이어 출력 확인 (디버깅용)
print(f"Layer {i+1} output shape: {x.shape}")
# 최종 정규화 (Pre-LN 구조에서)
output = self.final_norm(x)
return output, all_attention_weights
```

2. 각 레이어의 역할 분석

```
def  analyze_layer_representations():
"""각 레이어에서 학습되는 표현의 특성 분석"""
layer_characteristics = {
"Layer 1-2": {
"특징": "저수준 패턴 학습",
"학습 내용": [
"토큰 간 기본적인 관계",
"지역적 패턴 (n-gram 유사)",
```

```
"구문적 의존성의 기초"
]
},
"Layer 3-4": {
"특징": "중간 수준 패턴 학습",
"학습 내용": [
"구문적 구조 (주어-동사, 수식 관계)",
"의미적 역할 (주체, 객체, 술어)",
"중거리 의존성"
]
},
"Layer 5-6": {
"특징": "고수준 추상화",
"학습 내용": [
"복잡한 의미적 관계",
"장거리 의존성",
"문맥적 의미 표현",
"태스크 특화 표현"
]
}
}
return layer_characteristics
```

3. 레이어 수의 영향

```
def  compare_layer_numbers():
"""다양한 레이어 수에 따른 성능 비교"""
performance_analysis = {
"3 layers": {
"장점": ["빠른 훈련", "적은 메모리 사용"],
"단점": ["제한된 표현력", "복잡한 패턴 학습 어려움"],
"적용": "간단한 태스크, 리소스 제약 환경"
},
"6 layers": {
"장점": ["균형잡힌 성능", "대부분 태스크에 적합"],
"단점": ["중간 수준의 계산 비용"],
"적용": "일반적인 NLP 태스크"
},
"12+ layers": {
"장점": ["높은 표현력", "복잡한 태스크 처리"],
"단점": ["높은 계산 비용", "과적합 위험"],
"적용": "대규모 데이터, 복잡한 태스크"
}
}
return performance_analysis
```

완전한 사용 예시

```
def  demonstrate_encoder():
"""트랜스포머 인코더 사용 예시"""
# 하이퍼파라미터 설정
batch_size = 2
seq_len = 20
vocab_size = 10000
d_model = 512
num_heads = 8
d_ff = 2048
num_layers = 6
# 인코더 생성
encoder = TransformerEncoder(
num_layers=num_layers,
d_model=d_model,
num_heads=num_heads,
d_ff=d_ff,
dropout=0.1
)
# 입력 임베딩 레이어 (실제로는 토큰 임베딩 + 포지셔널 인코딩)
embedding = nn.Embedding(vocab_size, d_model)
# 더미 입력 데이터 (토큰 인덱스)
```

```
input_ids = torch.randint(0, vocab_size, (batch_size, seq_len))

# 임베딩 변환

x = embedding(input_ids) # [batch_size, seq_len, d_model]

# 패딩 마스크 생성 (예시)

# 실제로는 패딩 토큰의 위치를 0으로 설정

padding_mask = torch.ones(batch_size, 1, 1, seq_len) # 모든 토큰이 유효하다고 가정

# 인코더 순전파

encoder_output, attention_weights = encoder(x, padding_mask)

print(f"입력 크기: {x.shape}")

print(f"인코더 출력 크기: {encoder_output.shape}")

print(f"어텐션 가중치 개수: {len(attention_weights)} (레이어 수)")

print(f"각 레이어 어텐션 가중치 크기: {attention_weights[0].shape}")

return encoder_output, attention_weights


# 실행 예시

if __name__ == "__main__":

output, weights = demonstrate_encoder()
```

성능 최적화 기법

1. 메모리 효율성 개선

```
class EfficientEncoderLayer(nn.Module):
```

```
"""메모리 효율적인 인코더 레이어"""
def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
super(EfficientEncoderLayer, self).__init__()
self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)
self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)
self.norm1 = nn.LayerNorm(d_model)
self.norm2 = nn.LayerNorm(d_model)
self.dropout = nn.Dropout(dropout)
def forward(self, x, mask=None):
# 그래디언트 체크포인팅을 사용하여 메모리 절약
def attention_block(x):
norm_x = self.norm1(x)
attention_output, _ = self.self_attention(norm_x, norm_x, norm_x, mask)
return self.dropout(attention_output)
def ff_block(x):
norm_x = self.norm2(x)
ff_output = self.feed_forward(norm_x)
return self.dropout(ff_output)
# 체크포인팅 적용 (메모리 절약)
if self.training:
attention_output = torch.utils.checkpoint.checkpoint(attention_block, x)
x = x + attention_output
ff_output = torch.utils.checkpoint.checkpoint(ff_block, x)
```

```
output = x + ff_output

else:

attention_output = attention_block(x)

x = x + attention_output

ff_output = ff_block(x)

output = x + ff_output

return output, None
```

2. 계산 효율성 개선

```
def  create_padding_mask(input_ids, pad_token_id=0):

"""

패딩 마스크 생성 함수

Args:

input_ids: [batch_size, seq_len] - 토큰 인덱스

pad_token_id: 패딩 토큰의 인덱스

Returns:

mask: [batch_size, 1, 1, seq_len] - 어텐션 마스크

"""

# 패딩이 아닌 토큰은 1, 패딩 토큰은 0

mask = (input_ids != pad_token_id).unsqueeze(1).unsqueeze(1)

return mask.float()
```

```python
def  benchmark_encoder_performance():
"""인코더 성능 벤치마크"""
import time
# 다양한 설정으로 성능 측정
configs = [
{"num_layers": 3, "d_model": 256, "num_heads": 4},
{"num_layers": 6, "d_model": 512, "num_heads": 8},
{"num_layers": 12, "d_model": 768, "num_heads": 12}
]
batch_size = 16
seq_len = 128
for config in configs:
encoder = TransformerEncoder(**config)
x = torch.randn(batch_size, seq_len, config["d_model"])
# 파라미터 수 계산
total_params = sum(p.numel() for p in encoder.parameters())
# 실행 시간 측정
start_time = time.time()
for _ in  range(10):
with torch.no_grad():
_ = encoder(x)
end_time = time.time()
print(f"Config: {config}")
```

```
print(f"Parameters: {total_params:,}")

print(f"Time per forward pass: {(end_time - start_time) / 10:.4f}s")

print("-" * 50)
```

핵심 개념 정리

1. 인코더 레이어의 핵심 특징

- **셀프 어텐션**: 입력 시퀀스 내 모든 위치 간의 관계 모델링
- **피드포워드**: 각 위치에서 독립적인 비선형 변환
- **잔차 연결**: 그래디언트 흐름 개선과 학습 안정성
- **레이어 정규화**: 빠른 수렴과 안정적인 훈련

2. 6개 레이어 스택의 효과

- **계층적 학습**: 저수준부터 고수준까지 점진적 표현 학습
- **표현력 증대**: 복잡한 패턴과 장거리 의존성 포착
- **안정적 학습**: 잔차 연결을 통한 깊은 네트워크 훈련 가능

3. 실무 적용 고려사항

- **레이어 수 선택**: 태스크 복잡도와 리소스에 따른 조정
- **메모리 최적화**: 그래디언트 체크포인팅, 혼합 정밀도 훈련
- **하이퍼파라미터**: d_model, num_heads, d_ff의 균형잡힌 설정

다음 단계

인코더 아키텍처를 이해했다면, 다음 문서에서는 **디코더 아키텍처와 마스킹 메커니즘**을 살펴보겠습니다. 디코더는 인코더와 유사하지만 자기회귀적 생성을 위한 특별한 마스킹 메커니즘을 가지고 있습니다.

네비게이션

- **이전**: 멀티헤드 어텐션
- **다음**: 디코더 아키텍처
- **목차**: README.md
- **전체 목록**: 네비게이션

관련 문서:

- 기초: 어텐션 메커니즘
- 구성 요소: 멀티헤드 어텐션